

Dissemination of EUBUCCO

GEO5019: Final Engineering Report

by

Alexandre Bry
6277535

Carlo Cordes
6315895

Phuong Anh Ho (Alena)
6295488

30 January 2026

1. Introduction

This report describes the conversion and dissemination of the EUBUCCO dataset into three cloud-optimised geospatial formats: GeoParquet, FlatGeobuf and PMTiles. The code is available at <https://github.com/alexandre-bry/EUBUCCO-Dissemination>. It includes links to the data and to the website we made.

EUBUCCO is a comprehensive scientific database of the European building stock, with over 200 million individual building footprints across 27 EU member states and Switzerland. The dataset aggregates, harmonises, and validates data from over 50 open government datasets and OpenStreetMap. It provides three primary attributes: building height (73% of the buildings), construction year (24%), and building type (46%). The current dataset (v0.1) is officially hosted at <https://eubucco.com/>.

Despite the dataset's significant potential for urban sustainability studies, energy analysis, and more, its current distribution format presents substantial barriers for users. Currently, the data is fragmented into individual GeoPackage (`.gpkg`) and CSV files organised by country and only some cities. When working with EUBUCCO, users face three barriers:

1. **Accessibility:** There is no native support for querying specific sub-regions (e.g., a custom bounding box) without downloading entire country-level datasets.
2. **File Size:** A complete bulk download requires retrieving a single ZIP file that exceeds 100 GB, making it impractical for users with limited bandwidth or storage.
3. **Visualisation:** The current file formats lack cloud-native spatial indexing, making web-based visualisation slow.

To address these barriers, our project goal was to convert the original EUBUCCO data into cloud-optimised formats for both data visualisation and downloading. Additionally, we performed benchmarking of different formats to validate the efficiency and speed of the cloud-optimised formats compared to non-native cloud-optimised ones.

2. Methodology

Since the original EUBUCCO dataset (v0.1) consists of zipped GeoPackage files, we designed a process to convert these files into three specific formats:

1. **PMTiles:** For efficient, tile-based web visualisation.
2. **FlatGeobuf:** As an intermediate format for tile generation due to its fast encoding speeds and as a one of the offered download formats.
3. **GeoParquet:** For analytical queries and custom bounding box extractions due to its columnar architecture and metadata access.

To improve the accessibility of the EUBUCCO dataset and overcome the user's barriers, we divided the dissemination process into two parts:

1. **Visualisation:** By utilising MapLibre with a single PMTiles for the whole dataset allowing the users to explore the EUBUCCO dataset at multiple zoom levels without experiencing latency (detailed in *Chapter 3*).
2. **Downloading:** By providing both country-level and custom bounding box (bbox) download options in GeoParquet and FlatGeobuf format. For data to be downloadable, we pre-processed the data by partitioning it by country and by H3-index (detailed in *Section 4.a*). These two different partitions were then used for both static and dynamic approach:

- **Static Country Download:** Files partition by country in FlatGeobuf and GeoParquet formats (detailed in *Subsection 4.c.i*).
- **Dynamic Bbox Download:** A client-side spatial query engine using DuckDB-Wasm and H3-cell partitioned GeoParquet files. The system only retrieves the specific data files required for a user-defined bounding box (detailed in *Subsection 4.c.ii*).

The final phase of our methodology involved a benchmarking framework, detailed in *Chapter 5*. We evaluated the performance of GeoParquet and FlatGeobuf against the GeoPackage format across five tests: storage efficiency, row count, CSV export speed, attribute access performance, and spatial query efficiency. To assess scalability, these tests were conducted on three national datasets representing a spectrum of data and building densities: Cyprus (small-scale), Spain (medium-scale), and France (large-scale).

The newly cloud-optimised EUBUCCO dataset is hosted at <https://alexandre-bry.github.io/EUBUCCO-Dissemination/index.html>. The underlying EUBUCCO data were uploaded to S3 Object Storage.

3. Visualisation

3.a. Decisions and Output

In order to efficiently visualise the data, we decided to convert it into a single PMTiles file and use [MapLibre](#) to make the map on the website. To give clearer indications of the buildings covered by the dataset, we decided that the first few levels would display the boundaries of the countries instead of the buildings themselves, as they are too sparse to be clearly visible. This is shown in Figure 1.

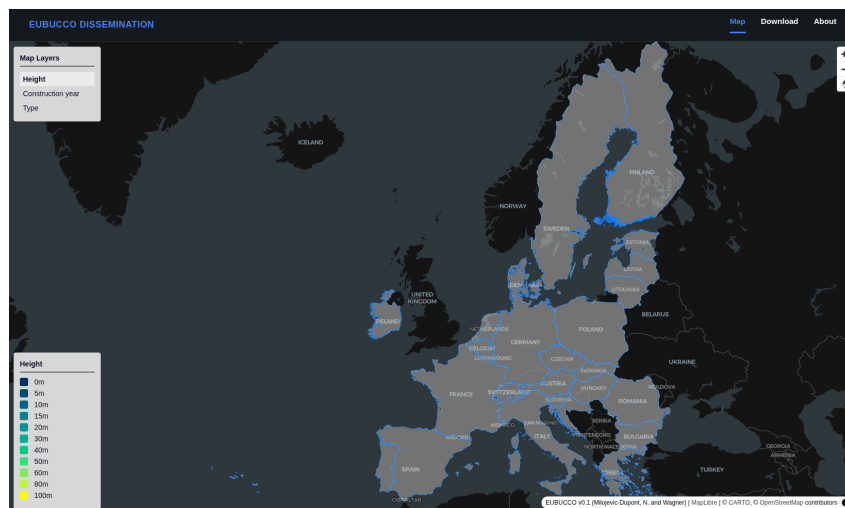
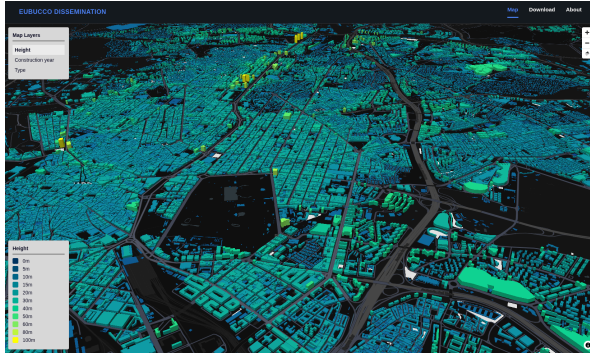


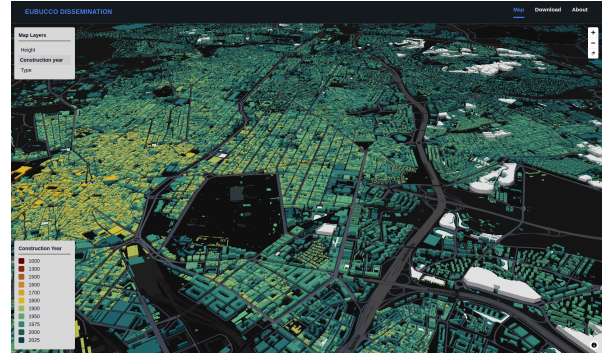
Figure 1: First view of the map when opening the website.

In terms of styling, we added buttons to pick which of the 3 available attributes (`height` , `age` or `type`) should be used to colour buildings, and buildings are represented as extruded polygons using their `height` attribute when it is specified. We use the [Dark Matter basemap from CARTO](#) as background, allowing to see missing buildings compared to the OpenStreetMap buildings. Finally, a legend was added to explain the colours.

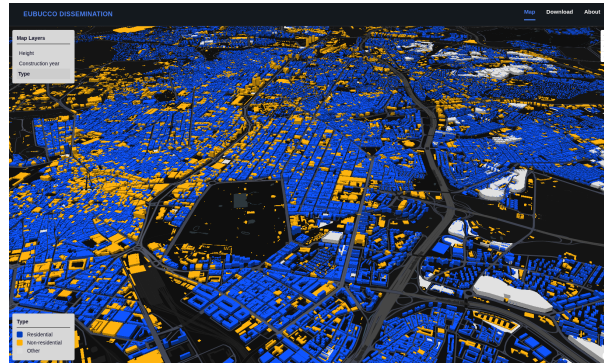
A great consequence of being able to visualise buildings from far away is the ability to see which countries or which regions of the countries actually have data for the different attributes.



(a) Coloured per height.



(b) Coloured per construction year.



(c) Coloured per building type.

Figure 2: Overview of Madrid in the map.

3.b. Process

We tried to build the simplest possible process to turn a list of zipped GeoPackage files into a single PMTiles. In order to actually create the PMTiles we decided to use [tippecanoe](#), a C++ library that produces PMTiles for vector data. However, PMTiles can only take as input GeoJSON, FlatGeoBuf and CSV, so we first had to convert the input formats into one of these. After making small experiments, it seemed that FlatGeoBuf was the quickest in terms of processing speed so we decided to use it as an intermediate. In the end, the final pipeline is the one displayed in Figure 3.

It is worth noticing that unzipping the GeoPackage files before converting them to FlatGeoBuf is crucial, as GDAL in general handles zipped GeoPackage files but at the cost of a much longer processing time. We made two experiments to for this, with the data from Cyprus (143MB zipped) and Czechia (1.6GB zipped). For Cyprus, conversion to FlatGeoBuf including unzipping took 10s instead of 20s by keeping the file zipped. For Czechia, the difference was 1min 51s *versus* 11min 28s.

Regarding administrative boundaries, we download them as GeoJSON from [geoBoundaries](#) using their API. These files are relatively small and used only for the lowest zoom levels so their impact on the size of the final PMTiles is negligible.

3.c. Potential Improvements

3.c.i. PMTiles Optimisation

A significant effort was put into trying to get the best possible result for the PMTiles, using the options provided by tippecanoe. As often, a balance needs to be found when generating them

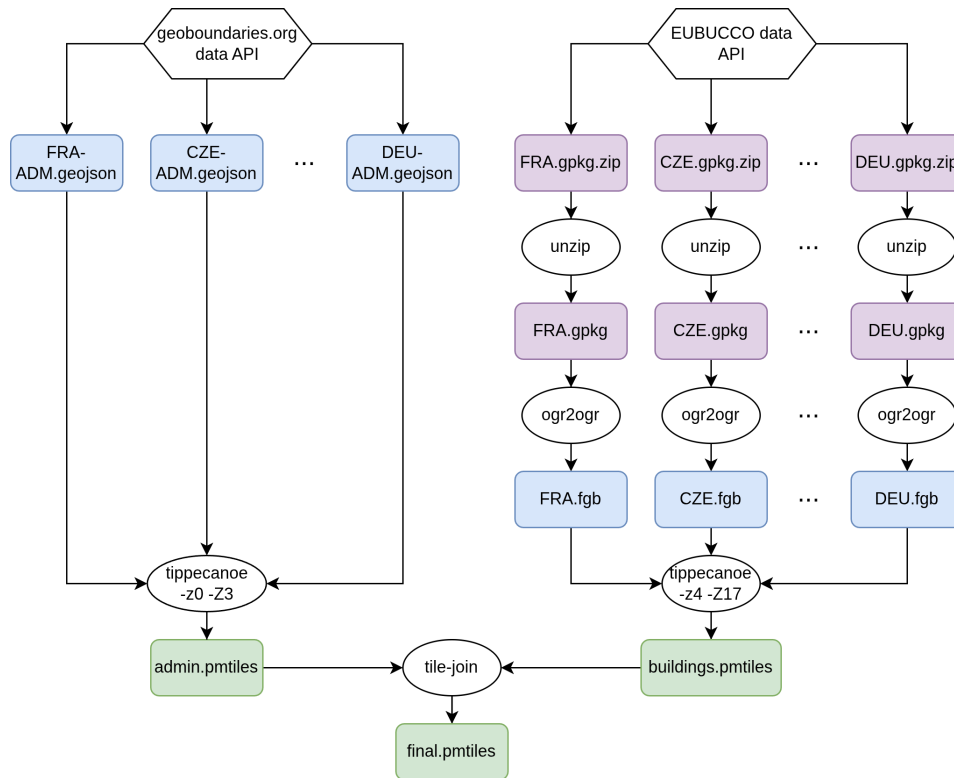


Figure 3: The pipeline to produce a single PMTiles from the input data.

between displaying as many buildings as possible and keeping the tile size small enough for quick queries. In our case, it is especially difficult because of the high density variations of the data between dense cities and the rest of the map.

The main ways of reducing the size of tiles for polygons are dropping them or coalescing them. Coalescing means merging the nearby polygons to reduce the feature count and the size of the polygons. In both cases, the polygons can be picked using different criteria: the smallest polygons, the densest polygons or a fraction. In order to keep as much as possible the surface spanned by the buildings visible at lower zoom levels, coalescing seems like the best option. However, coalescing is intended for polygons that share boundaries, which is often not the case for buildings, which can have space between them. The line simplification algorithm can also be boosted using the `--simplification` option with a value higher than 1, in order to get simpler polygons which are therefore smaller in size.

The parameters we ended up using are the following:

- `-z4` and `-Z17` for buildings to limit the zoom levels of buildings between 4 and 17. Limiting the zoom levels to a maximum of 17 (8 cm precision) reduces the size of the final file since the final tiles are the most precise and largest ones.
- `--drop-smallest-as-needed` to try reducing the size of tiles but keep the largest and most visible objects,
- `--simplification=5` to increase line simplification and get smaller polygons in size
- Arguments to only keep the attributes we want to show:
 - `--include=height`,
 - `--include=age`,
 - `--include=type`,
- Attributes to control the size of the final tiles:

- ▶ `--no-feature-limit` : no limit for the number of features per tile, we limit things with the size of the tiles,
- ▶ `--maximum-tile-bytes 1000000` : a maximum of 1MB per tile.

The 1MB limit for tiles was picked arbitrarily. The default value is 500kB, meaning that our tiles can be up to 2 times bigger than the default maximum size, resulting in longer visualisation times, both because of the tile size and because of the number of features to load in the map. Increasing the value was necessary in order to prevent some zoom levels from looking very empty. However, when switching storage from Hetzner to Source Cooperative, the downloading speed of the tiles was dramatically reduced, multiplying the time to download by up to 20 times.

3.c.ii. Processing Speed

It must be noted that running the whole pipeline currently takes a significant amount of time. The process of making one PMTiles out of the buildings of all countries as FlatGeoBuf takes about 5 hours when running on gilfoyle, with about 60 hours of cumulative CPU time thanks to tippecanoe multi-processing. However, we do not know how the processing speed of tippecanoe can be improved. It may seem interesting to run every country separately and then merge them together, however running them separately means that the tile limits will be applied individually to every country. The two main consequences are that a tile spanning over several countries could end up much larger than the specified limit, and the density of the buildings dropped in each country will be different as the densest and largest one will not require the same levels of reduction.

Then, the fact that GDAL is mostly single-threaded is also a significant limitation for our pipeline. Every country gets assigned the same amount of computing power (one core), even though the largest one (Germany) has a size of 76GB as unzipped GeoPackage, *versus* 41MB for Luxembourg. Even Italy, the second largest country is data size, “only” represents 22GB. Therefore, the processing of Germany is the actual bottleneck of the pipeline. This is something that could maybe be improved by splitting the countries into smaller files when necessary. For example we could aim for having files of at most 1GB (which would result in slightly more than 200 files), and then have one process for each of them. This solution could help making the best of the 128 available cores. Then, all the resulting FlatGeoBuf files could be given as the input of tippecanoe. Due to time constraints, we have not tried to implement this.

3.d. Better Basemaps

Another aspect that could really improve the current visualization is to improve the basemaps. A simple thing could be to provide more options for basemaps. But a more advanced and more interesting improvement would be to isolate the text displayed by a basemap, and have it displayed on top of the buildings, to ensure that the text is still visible despite the 3D buildings.

4. Downloading

To ensure the EUBUCCO dataset is accessible to a wide range of users requiring either country-scale data or local data, we implemented a dual-option downloading system on the website. This system distinguishes between static retrieval for country-level datasets and dynamic, client-side processing for custom bounding box regions.

4.a. Data Preparation

When using the original EUBUCCO page, we came to the conclusion that we wanted data available in two different formats. Users looking to investigate data on a specific country should have their needs catered to as much as one looking for a specific area that might cross borders. We therefore opted to host this data twice separately, to fulfil these needs. The algorithms for the following section developed locally and then deployed in parallel on Gilfoyle. The transfer of data between Gilfoyle and the storage was carried out using `boto3`.

4.a.i. Country Partition

As the raw data came delivered as zipped gpkg-files per country, this was perhaps the easiest to implement. The conversion was done using `geoparquet-io`. It handles geospatial data types natively and implements best practices for storage of geoparquet natively:

- `zstd` for compression, compression level: 15.
- Include `bbox`
- GeoParquet version 1.1.
- Spatial ordering along a Hilbert curve
- Maximum row group size between 50,000 and 150,000 per row
- If the data is larger than 2 gigabytes consider spatially partitioning the files

The sole exclusion from this list is that the files were not partitioned to the optimal 2Gb size, as the layout of our goal here does not allow for this. The resulting files per country are quite diverse in size ranging from 17Mb (Luxembourg), to 4Gb (Austria) and 43Gb (Germany). Most files however meet this specification with a size between 100Mb and 1.5Gb.

4.a.ii. H3 Partition

The H3 partition schema is global and hierarchical, originally developed to serve the needs of Uber for global navigation and location services. In principal, the surface of the globe is divided into many hexagons and (for geometry reasons) some pentagons. Each of these shapes is then partitioned into further hexagons. This process is carried out iteratively to yield geometries on 16 different levels, also called resolutions. On the coarsest level, level 0, the globe is partitioned into 122 cells. At level 15, the finest, we find 569,707,381,193,162 partitions. Each of the cells at different levels are assigned a semantic descriptor (e.g. `841ee21fffffffff`) which serve as the unique identifier of a cell. Further important to this is that these indices are also structured in hierarchy. This means that for every given index we can concretely identify with which other cells an index might share area. This might be the case with cells of different resolution. A cell with lower resolution is called a parent index, cells with shared area with higher resolution are called children. In principle, this is a simple tree structure.

To apply this to geographical partitions, this setup allows to structure building data into geometries dictated by h3-indices and therefore obtain a very localised and consistent partition of our data. Again `geoparquet-io` has a nice implementation that carries out partitions like these. Using the country partitioned geoparquets as inputs, we scan over each file individually and bin them into containers of h3-index. The results of this are then consequently stored as parquet files dictated by their respective index-code. A folder structure which promotes pruning will querying was chosen as such: `h3_cell=841ee21fffffffff/841ee21fffffffff.parquet`. In this investigation we chose to partition the files at level 4, which resulted at roughly 3000 separate files. This was coarse enough to have a substantial amount of features in one file and not too fine so that a query would have to open too many files at once and being detrimental to

query speed. Partition sizes of the resulting files were more consistent than for the country partitions. The average file size is around 20Mb.

4.b. Object Storage

To host our data, we chose to open an object storage on `Hetzner.com`, which mirrors an S3 bucket. We settled for the cheapest option of having just object storage and no compute as we would be doing most of the processing client side. The cheapest option came with 1TB of storage along with 1TB of traffic. The pricing runs at roughly 5€ per month. This bucket could be directly accessed using boto3 from either a local machine or the gilfoyle server to transfer files.

We eventually replaced this service with `source.coop` as it allows us to host data publically for free, while having slightly less throughput than Hetzner.

4.c. Data Download Implementation

The web-page was designed to facilitate two use cases: downloading entire country datasets and extracting data for a custom bounding boxes (bbox).

4.c.i. Country Download

The implemented country download feature provides access to full country datasets in two cloud-native formats: FlatGeobuf and GeoParquet.

When the user selects a country and a preferred format, the application triggers a direct HTTP retrieval of the corresponding file from the S3 object storage. We decided on static hosting approach since we encountered limitations with dynamic in-browser processing using DuckDB-Wasm.

Initially, we attempted to use a live DuckDB-Wasm connection to fetch and filter these national files dynamically. This approach was successful for smaller datasets such as Bulgaria or Cyprus (~163.68 MB). However, it proved to be inefficient for larger countries such as Germany (~43.69 GB). Loading a GeoParquet file of this size into the browser's memory consistently triggered memory allocation errors, because it exceeded the memory limits available to the WebAssembly.

Consequently, we implemented a pre-partitioned static retrieval system. The application receives the user's selection country in the ISO 3166-1 alpha-3 country code (e.g. `AUT` for Austria) and constructs a direct GET request URL (e.g., `.../partition-country/AUT.parquet`), ensuring successful downloading functionality regardless of the file size.

4.c.ii. Bbox Download

Unlike the country download, the implemented bbox download was handled dynamically. Additionally, it is only offered in GeoParquet format. Once the user provides the bbox coordinates (either manually or by drawing), the data retrieval process is then divided into 4 steps:

Step 1: DuckDB-Wasm Deployment and Setup

The download process begins by initialising the DuckDB-Wasm engine. The application selects the appropriate WebAssembly bundle for the user's browser capabilities (MVP or EH) and instantiates a Web Worker to ensure the browser user interface (UI) remains responsive during processing. Once the database is instantiated, the `https` and spatial extensions are loaded.

We also configure the database environment by setting S3 parameters (such as endpoint, region, and SSL) to allow DuckDB to communicate directly with our object storage.

Step 2: From Bbox Coordinates to H3-Cells Mapping

The inputted coordinates are received as `minLon`, `minLat`, `maxLon`, `maxLat` which are converted into a polygon using `h3-js` library, specifically `polygonToCells` function at resolution 4. The function calculates which H3 cells at resolution 4 are needed to cover the entire area inside the provided bbox (polygon). This is returned as array of strings, each string is a unique H3 cell ID (e.g. `['841f1d5fffffffff', '841f1d7fffffffff', ...]`).

However, we encountered an issue when only using `polygonToCells` function. Since H3 cells at resolution 4 are quite large ($\sim 1,770\text{km}^2$), a small bbox (e.g. for city Sofia, Bulgaria) could be entirely located within one H3 cell, without even intersecting the cell's centre. This resulted in `No building data found` error because `polygonToCells` only returns H3 cells IDs whose centre points fall within the bbox. To solve this, the mapping logic was improved by also using `latLngToCell` function, which retrieves the H3 Cell IDs for the four corners and the centre point of the bbox.

These sets are merged to ensure that even small bounding boxes correctly identify their containing H3 cell. We only keep the unique H3 cell IDs. These H3 cell IDs directly correspond to the partitioning scheme in our S3 storage. We then constructed S3 URL paths to the necessary GeoParquet files (e.g. `s3://.../h3_cell=841f1d5fffffffff/841f1d5fffffffff.parquet`). This mapping eliminates the need to request all available H3-GeoParquet files (3197) from our S3 bucket, but we only target the specific files that are relevant for the bbox.

For example, if we choose to download Zuid-Holland (red-lined) as our hypothetical bbox, we would only need eight H3-GeoParquet files, at resolution 4 (Figure 4).

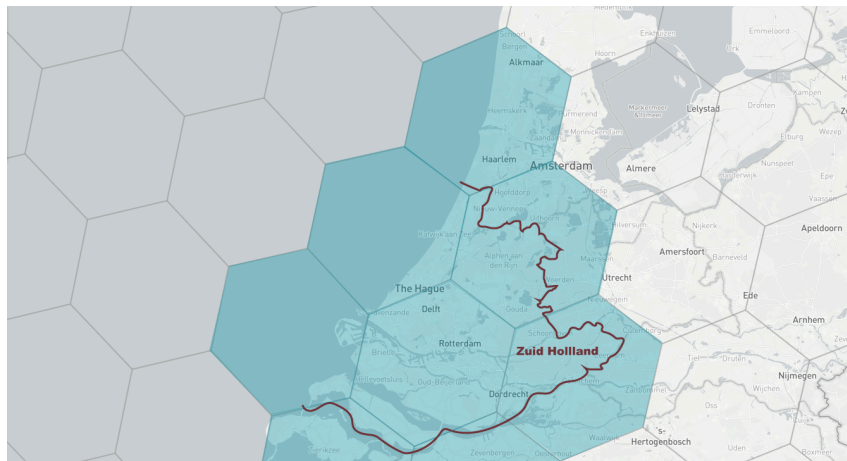


Figure 4: Highlighted H3 tiles needed for Zuid Holland bbox

Step 3: H3-GeoParquet Files Pruning

The obtained list of necessary H3-GeoParquet files for the required bbox is then cross-referenced with the a pre-generated `manifest.json`. This is to prevent the client from requesting non-existent files, since the manifest contains a verified list of all H3 cells that are in our S3 bucket (i.e. EUBUCCO data). The manifest is created via a pre-processing script that crawls the S3 bucket and lists all unique H3 cell IDs that contain data (~ 59.32 KB). After the creation, the manifest itself is uploaded to the S3 bucket and then retrieved via URL path during the actual downloading process. By fetching the manifest first, we can immediately filter the search area before making any large network requests for the GeoParquet files.

Once the final list of valid files is obtained, we also check the number of files. Since each H3 cell corresponds to a separate GeoParquet file in the S3 bucket, we decided to create a limit of how many files can be retrieved at once. If the user selects an area encompassing more than 200 H3-files, the application interrupts the process. This was necessary because establishing HTTP range-requests for hundreds of individual files would create significant network overhead and latency. In such cases, the user interface (UI) advises the user that the area is too large and suggests downloading the whole country files or selecting a smaller bbox instead (Figure 5).

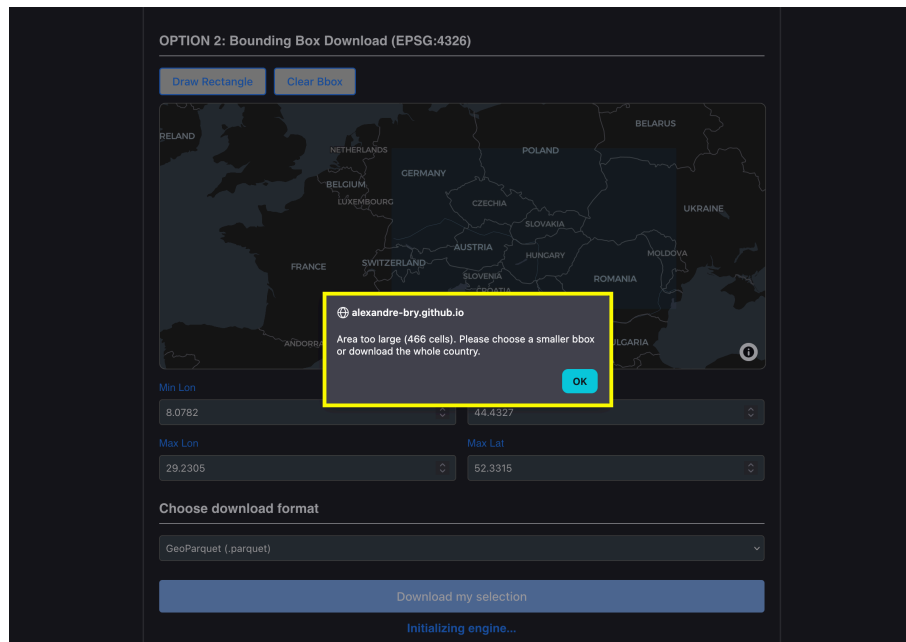


Figure 5: UI message when selecting a very large bbox

Additionally, if the returned list of valid files is empty, it indicates that while the user selected a bbox, no building footprints exist in our dataset for that region (e.g., selection over the sea or a non-mapped area). Therefore the UI immediately informs the user and stops the process (Figure 6).

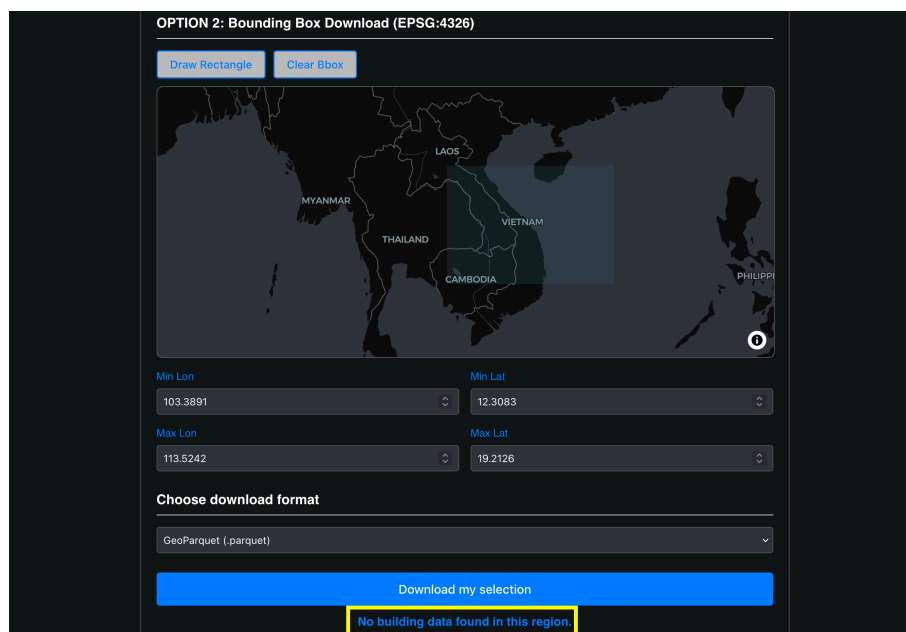


Figure 6: UI message when selected empty/unmapped area

Step 4: SQL Extraction

The last step of the process is the query execution of `ST_Intersect` using `ST_MakeEnvelope` which clips the H3 tiles to the requested bbox. The resulting buildings are then moved from the DuckDB virtual system to a JavaScript buffer and delivered as a GeoParquet file which is downloaded to the user's system.

Wasm Limitations

During the implementation testing, selecting high-density urban areas or larger bboxes containing a significant number of buildings often resulted in `Out of Memory Error: Allocation failure`.

Therefore, to ensure downloading capabilities stability as much as possible, we implemented two checks:

1. **Metadata row-count:** Before extracting the buildings data immediately, we first query the GeoParquet metadata with `parquet_metadata` function and get the total number of rows (buildings) without loading the building geometries into the browser memory. We established a threshold of 2 million buildings. If the metadata check returns a value higher than this, the application interrupts the process and informs the user via the UI message (Figure 7). This prevents the browser from running a query that would lead to a memory allocation failure.
2. **Wasm memory clean-up:** After the successful query execution, we implemented a clean-up step using `db.dropFile()`. As soon as the data is copied into the JavaScript buffer, the temporary file is then deleted from the DuckDB virtual disk resulting in release of memory.

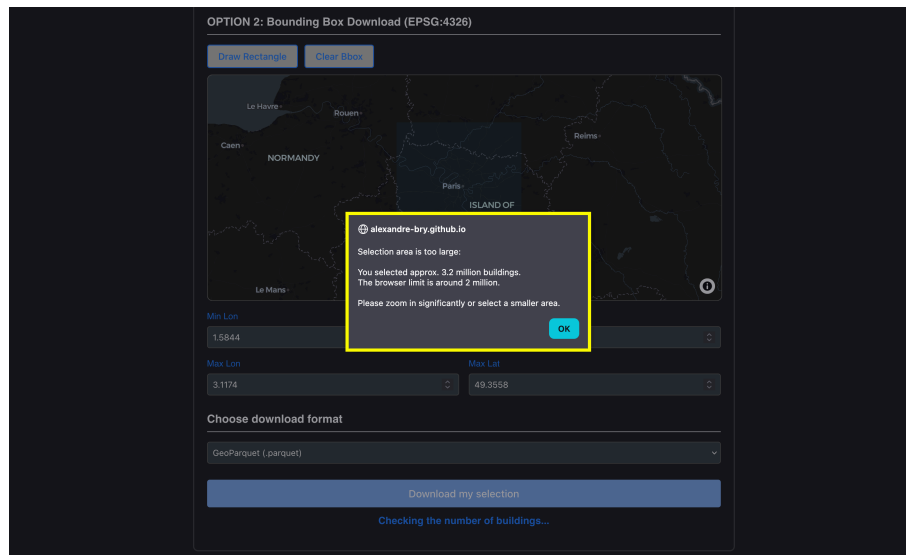


Figure 7: UI message when trying to download more than 2 million buildings

However, it is important to note that these checks do not overcome the errors. Since the memory consumption depends on other factors, such the complexity of individual geometries rather than just the row count, allocation errors can still occur if the memory becomes fragmented or if the peak memory usage during the spatial intersection exceeds the browser's limit. In our experience refreshing the page and disabling/deleting cache helps to reset the environment.

4.c.iii. User Interface Features

The country download and bbox download have different available downloading formats. Therefore, we designed the UI in such a way to prevent users from accidentally triggering conflicting download types. Additionally, the bbox download provides an interactive coordinates selection by drawing on a map. The whole downloading process is then accompanied with visual feedback messages.

Downloading type exclusivity

To guide the user toward a valid request, we implemented the following visual logic:

1. **Country vs. BBox:** Selecting a country automatically clears the bbox inputs and disables them (setting their opacity). Conversely, entering coordinates or drawing on the map disables the country selector.
2. **Format Visibility:** While country downloads support both FlatGeobuf and GeoParquet, the bbox download is restricted to GeoParquet. The UI enforces this by disabling the FlatGeobuf option and updating its label to `FlatGeobuf (Country only)` whenever bbox coordinates are detected.

Interactive BBox Drawing

The bbox selection is also supported by an interactive map using `MapLibre GL` and the `mapbox-gl-draw-rectangle-mode` plugin.

- **Rectangle Tool:** When the user clicks `Draw Bbox`, the map enters a drawing mode. Upon completion, the map's coordinates are automatically calculated and inserted into the bbox input fields. If the user is not satisfied with the drawn bbox, it can be removed when user clicks on `Clear Bbox` to restart the drawing process.
- **Bi-directional Option:** We designed the UI to be flexible. Therefore, the user can either draw on the map or manually type coordinates into the inputs. The map and manual coordinate inputs stay synchronised by using the event listeners.

Visual Feedback

During the bbox extraction process, the `Download` button is disabled to prevent duplicate requests. The status message area below the button provides feedback. If an error occurs (such as a memory overflow or a network timeout), the error is caught and displayed directly in the UI, ensuring the user is informed. This can be seen in the Figures above throughout the *Section 4.c*.

5. Benchmarking and Format Comparisons

To validate the efficiency of the cloud-optimised formats, we conducted a series of benchmarks. The goal was to quantify and validate the theoretical improvements of cloud-optimised formats. We divided the testing into two distinct areas of focus:

1. **Local Performance Benchmarking:** These tests, conducted on a local environment, focus on the computational and I/O efficiency of the formats. We measured how the internal structure of each format affects CPU usage, memory allocation, and disk read speeds during the analytical tasks.

2. **Online Query Benchmarking:** These tests focus on evaluating the partitions. By hosting the files on a remote S3 bucket and querying them via DuckDB, we measure the cloud-native formats' capabilities to transfer random bounding boxes quickly.

5.a. Local Performance Benchmarking

5.a.i. Used Device

To ensure consistency, we performed the benchmarks on one device only. Specifically, on MacBook Pro 16-inch (2024 model) with the following specifications:

- Processor: M4 Pro chip (14-core CPU with 10 performance cores and 4 efficiency cores).
- Memory: 24 GB Unified Memory

5.a.ii. Benchmarking Methodology

Using DuckDB and Python, we compared the following four file formats:

1. **GeoPackage** (`gpkg`)
2. **Zipped GeoPackage** (`gpkg.zip`)
3. **GeoParquet** (`.parquet`)
4. **FlatGeobuf** (`.fgb`)

To observe how these formats scale, we then performed the benchmarking tests on three selected countries with varying building densities :

- **Small Scale:** Cyprus (~468 thousand buildings)
- **Medium Scale:** Spain (~16.3 million buildings)
- **Large Scale:** France (~47.8 million buildings)

The benchmark consisted of five distinct tests designed to stress different aspects of I/O, compression, and query execution:

- **Test 1. Storage Size Comparison:** The measured physical disk size of each file in megabytes (MB).
- **Test 2. Row Count Speed:** The time required to execute a `SELECT count(*)` query on the full dataset. We aimed to test the efficiency of the format's metadata or index reading capabilities. Columnar formats (like Parquet) often allow retrieving counts from metadata without scanning the dataset.
- **Test 3. CSV Export Speed (Full Dataset Processing):** The time required to convert and export the entire dataset to a CSV file. This was done with a query selecting all attributes and converting the binary geometry column to Well-Known Text (WKT) using `ST_AsText()`.
- **Test 4. Attribute Access Performance:** The time required to perform statistical calculations on a non-spatial numerical column (`height`, the only attribute present in all selected countries). For this two queries were executed: `MIN(height)` and `MAX(height)` and `AVG(height)`.
- **Test 5. Spatial Bounding Box (BBox) Filtering:** Measuring the efficiency of spatial indexing by retrieving buildings that intersect a random bounding box. We performed spatial intersection tests using `ST_Intersects` against a dynamically generated bounding box coordinates. We tested three window sizes (500m, 5km, and 20km) and for each size we generated 5 random bounding boxes within the dataset's total extent. The reported time is the average of these 5 repetitions.

- **Constraint:** For the larger datasets (Spain and France), the Zipped GPKG format was excluded from Test 2 - 5 to prevent excessive execution times caused by repeated decompression.

5.a.iii. Benchmarking Results

Disclaimer: for readability reasons, many of the figures displayed in this section use a logarithmic scale on the vertical axis.

Test 1: Storage Size Comparison

The storage size comparison (Figure 8) showcases that GeoParquet is consistently the most space-efficient format for large datasets. In the larger country datasets (Spain and France), GeoParquet achieved a significantly smaller disk footprint than even the Zipped GeoPackage. However, for the smallest dataset (Cyprus), the Zipped GeoPackage (142 MB) remained slightly more compact than GeoParquet (164 MB).

For Spain, the GeoParquet (2.58 GB) was around 60% smaller than the uncompressed GeoPackage (6.56 GB) and FlatGeobuf (6.36 GB). Additionally, France achieved similar results, with GeoParquet (5.28 GB) versus GeoPackage at (15.25 GB).

While Zipped GeoPackage achieves compression ratios similar to GeoParquet (e.g., 2.63 GB for Spain), it is functionally unusable for direct analysis due to its decompression overhead we observed in subsequent tests.

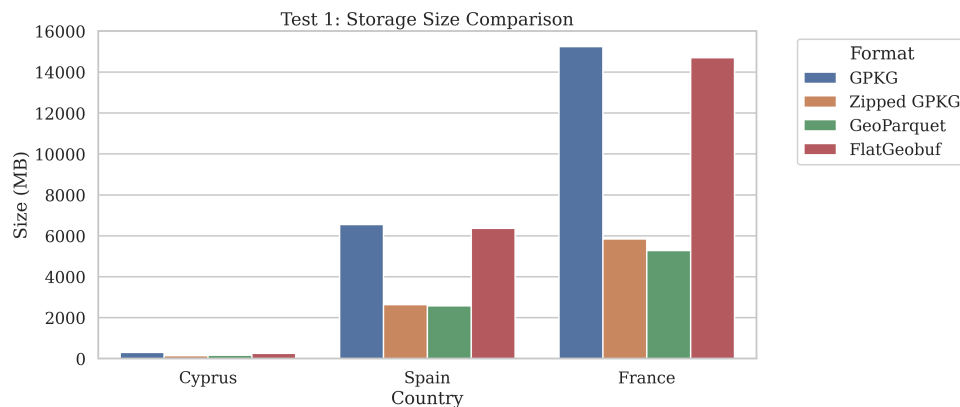


Figure 8: Graph showing results of Test 1

Test 2: Row Count Speed

This test measured the time required to determine the total feature count (e.g., number of buildings). As shown in Figure 9, GeoParquet is the fastest performer across all scenarios, completing the task in ~0.001 seconds regardless of file size. This is possible because GeoParquet stores row counts in its metadata footer. Therefore, DuckDB can retrieve the count without scanning the data first.

In contrast, Zipped GeoPackage was the worst performer. Even for the smallest dataset (Cyprus), it required over 7 seconds to decompress and count features. Due to this, we decided to exclude Zipped GeoPackage from the larger dataset tests.

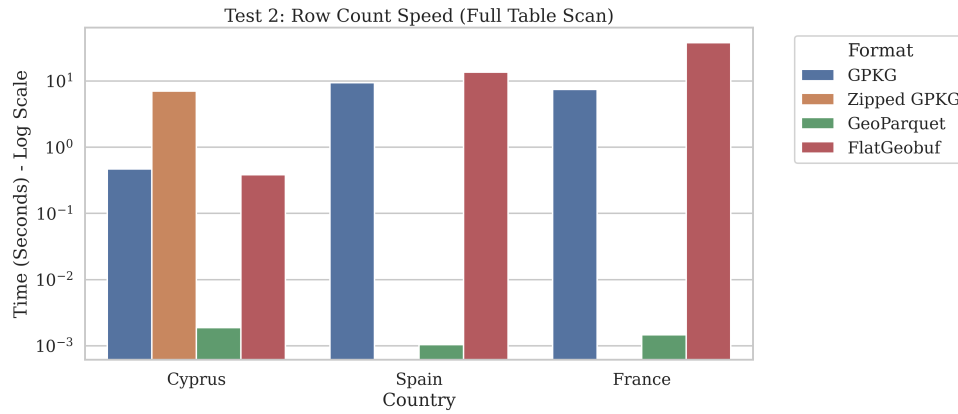


Figure 9: Graph showing results of Test 2

Test 3: CSV Export Speed

This tested the full-table read throughput. For the small Cyprus dataset, GeoPackage (~ 1.56 s) and GeoParquet (~ 1.57 s) performed almost identically. However, performance differed as data scale increased (Figure 10).

For Spain, GeoParquet was fastest (~ 16.8 s), significantly outperforming GeoPackage (~ 31.0 s) and FlatGeobuf (~ 44.5 s). However, for France, GeoPackage (~ 65.3 s) slightly faster than GeoParquet (~ 77.3 s).

While Parquet is optimised for column reads, exporting to CSV requires re-writing every row and serialising geometries to WKT. In this case columnar architecture can be a disadvantage. This could explain the mixed performance of GeoParquet. Finally, FlatGeobuf consistently performed least optimally in this test.

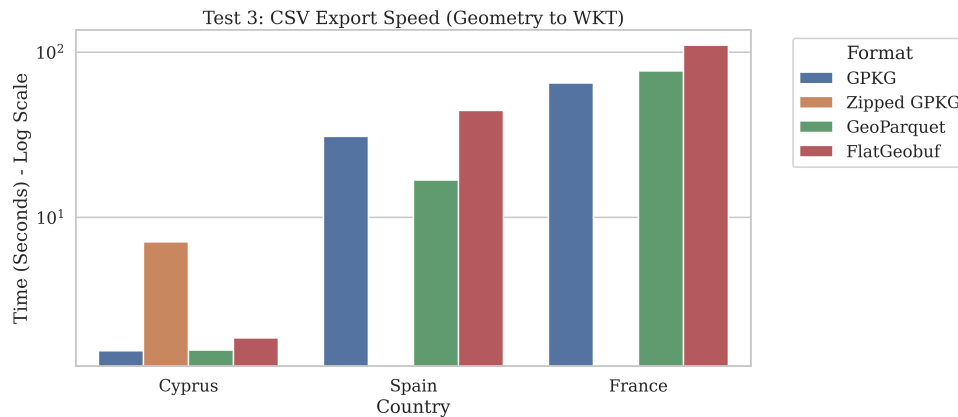


Figure 10: Graph showing results of Test 3

Test 4: Attribute Access

The results of this test highlight the columnar architecture of GeoParquet. For `AVG(height)`, it only needed to read the specific `height` column rather than the entire row.

Therefore, for the France dataset, retrieving `AVG(height)` took ~ 0.017 seconds with GeoParquet. This is orders of magnitude faster than GeoPackage (~ 15.9 s) and FlatGeobuf (~ 44.2 s). These formats must perform full table scans (Figure 11).

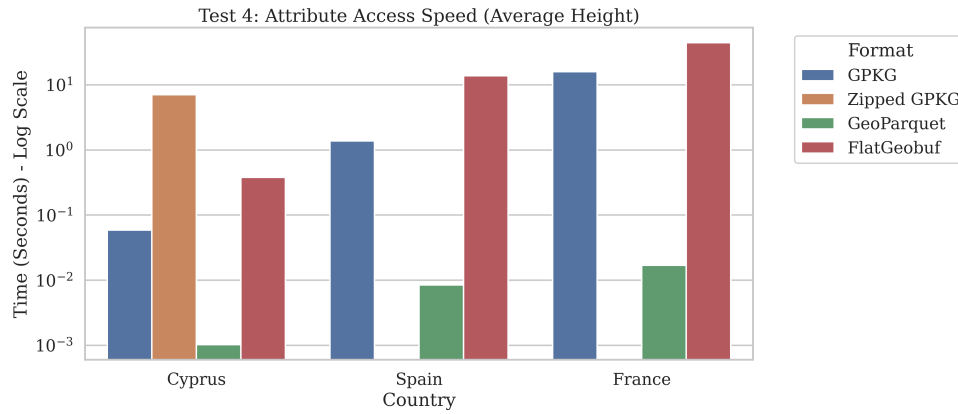


Figure 11: Graph showing results of Test 4 - Average Height

As for Min/Max Calculations, a similar pattern emerged for finding minimum and maximum values (Figure 12).

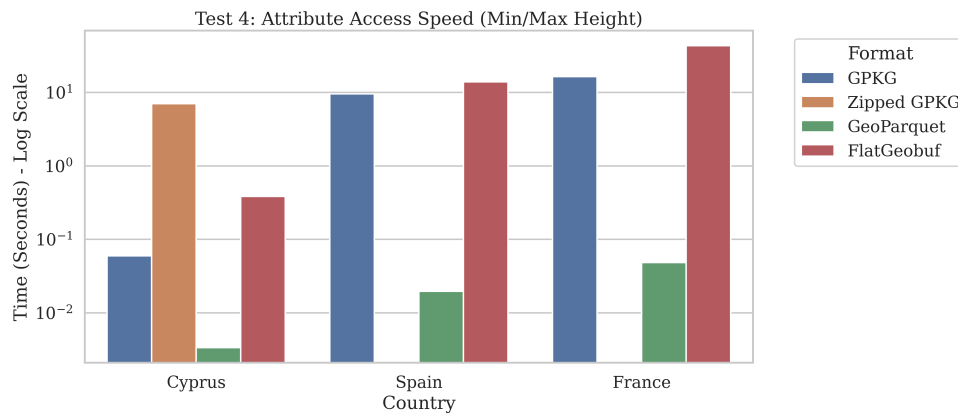


Figure 12: Graph showing results of Test 4 - Min/Max Height

Test 5: Spatial BBox Filtering

Theoretically, we expected FlatGeobuf to perform the best in this test due to its spatial index. However, our results show it was the slowest format (excluding Zipped GeoPackage), taking ~ 52 seconds for the France dataset compared to ~3.3 seconds for GeoParquet.

When looking at the query times (Figure 13), we can see that they are nearly identical regardless of the bounding box size (500m vs 20km) for all formats. This could indicate that full scans were performed. This suggests that the DuckDB `spatial` extension also performs a full table scan for FlatGeobuf rather than utilising its spatial index.

Therefore, GeoParquet performed fastest because of its high read throughput which allows for faster geometry column scan compared to other formats.

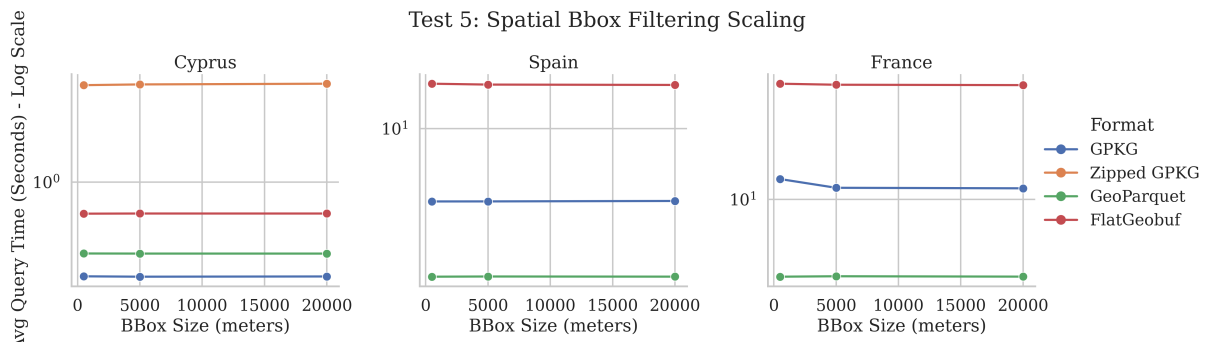


Figure 13: Graph showing result Test 5

5.a.iv. Local Benchmark Conclusion

Overall, GeoParquet seems to be the most suitable format for analytical tasks. Its ability to store metadata statistics (Min/Max/Count) allowed it to answer attribute queries time-efficiently. Furthermore, it consistently offered the most efficient uncompressed storage footprint. It achieved ~60% reduction in file size compared to GeoPackage for the Spain and France datasets.

Contrary to our expectations, the spatially indexed FlatGeobuf format underperformed in bounding box queries. This could suggest that the current DuckDB spatial extension does not yet utilise FlatGeobuf's spatial index, instead it uses a full table scan. However, it is important to note that our inclusion of FlatGeobuf was driven by its role in our visualisation pipeline (*Chapter 3*), specifically for generating PMTiles. While FlatGeobuf underperformed in DuckDB-based analytical queries compared to GeoParquet, it remains useful for visualisation purposes.

As for GeoPackage, it was mostly slower for analytical tasks. Its row-oriented architecture requires full-file scans even for simple row count. However, it was competitive for files export tasks (CSV conversion) where row-by-row reconstruction was necessary.

Finally, the Zipped GeoPackage offers compression for archival purposes but is functionally unsuitable for direct analysis due to extremely long decompression times.

5.b. Online Query Benchmarking

This part of benchmarking investigates the value of the partitions to the user download. Specifically, we want to determine the worthiness of a service to deliver files by request of a bounding box in the two partition scenarios we have provided: h3 and per-country.

5.b.i. Methodology

In this section we created a bounding box test which would iteratively query the two endpoints (h3 and country) with the same geometry, time it and see how the two compare. In essence the methodology used looks like this:

1. Create randomised set of bounding boxes (300)
2. Prune specific file paths, in order not to account for time spent scanning
3. Timed range request for both partitions
4. Append results

There are a few special things to point out about this process. We wanted to introduce another variable to this equation which is the bounding box size. The center points of the bounding

boxes are selected randomly, however the sizes are predetermined. The testing set comprises 10 categories of sizes up to 2 by 2 degrees in size. Each category is tested 30 times. We do this to investigate whether there is a point at which one strategy surpasses the other. In order not to measure the time in which we spend scanning the metadata of each file on the object storage, we do this process locally. Having done this, we pass the specific endpoint URLs required to DuckDB from which we retrieve the data. We record the time taken for the request, along with the number of files opened and the amount of features for each bounding box. In theory we should have identical results for each trial.

It is further important to note that Germany had to be excluded from the exercise due to its file size. The file consistently caused memory issues during the requests and made range requests within Germany extremely slow. We therefore settled for a study area that ranged from southern Italy to northern Poland (Warsaw).

5.b.ii. Results

In summary, h3 partitioning outperforms the by-country application in every aspect possible. Figure 14 shows the average query speed for 10 different sizes of bounding boxes for both methods. While being extremely consistent, the h3-partition shows that even for larger sizes of bounding boxes, randomly chosen data is delivered within a second. Perhaps most notably, the country partition struggles to deliver small regions in particular.

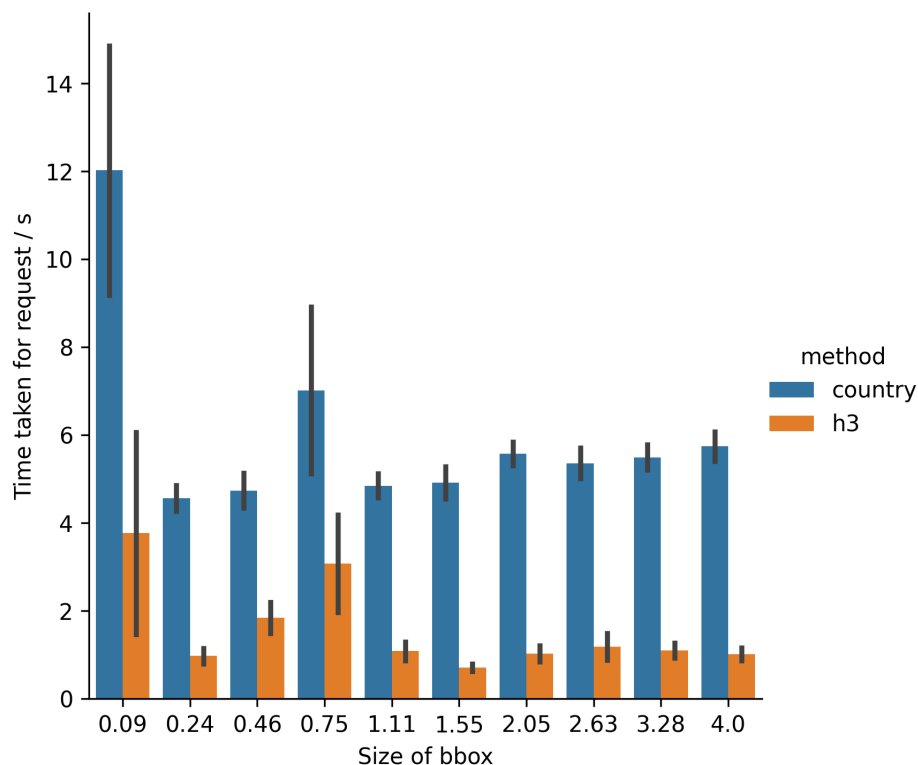


Figure 14: Graph showing time taken of query by varying bounding boxes

We also tested the relationship of the query speed against the number of files we needed to open for the delivery to take place. Figure 15 demonstrates clearly, that despite the need to go through sometimes 30 or more files, the delivery is still faster than when executing the same query on larger files. The by-country partitions show a linear trend with the delivery time, each added file adding complexity and time to the query.

The same picture is painted when examining the number of features returned against query speed. The spatially heterogeneously organised country partition shows quite a diffuse result, with delivery times varying quite larger. The h3 counterpart on the other hand demonstrates a very consistent pattern, consistently being the faster alternative. The investigation showed that up to 250,000 features could still consistently be delivered within 5 seconds.

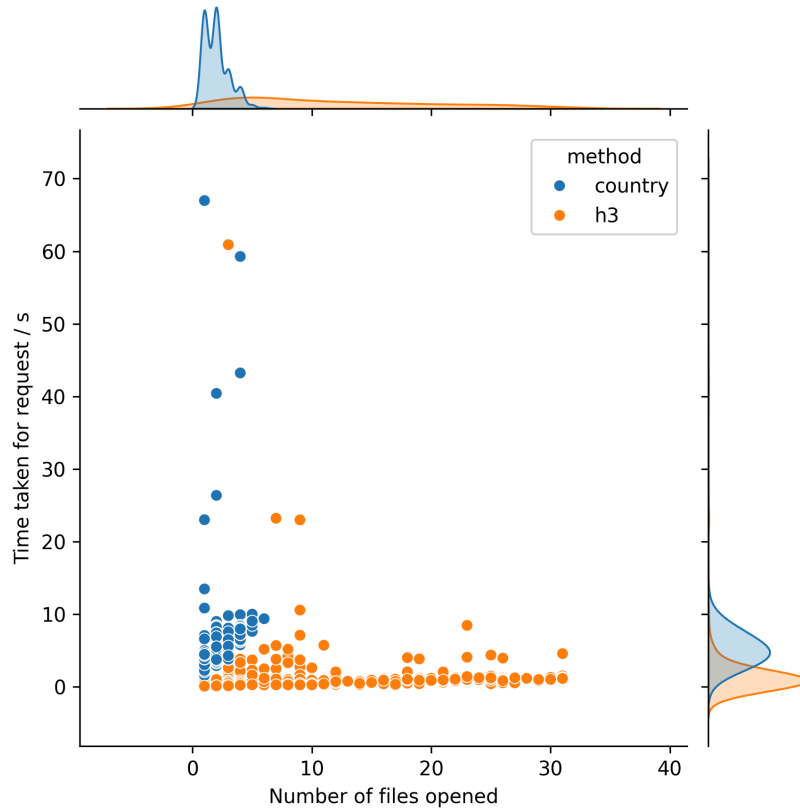


Figure 15: Graph showing time taken of query by number of files opened

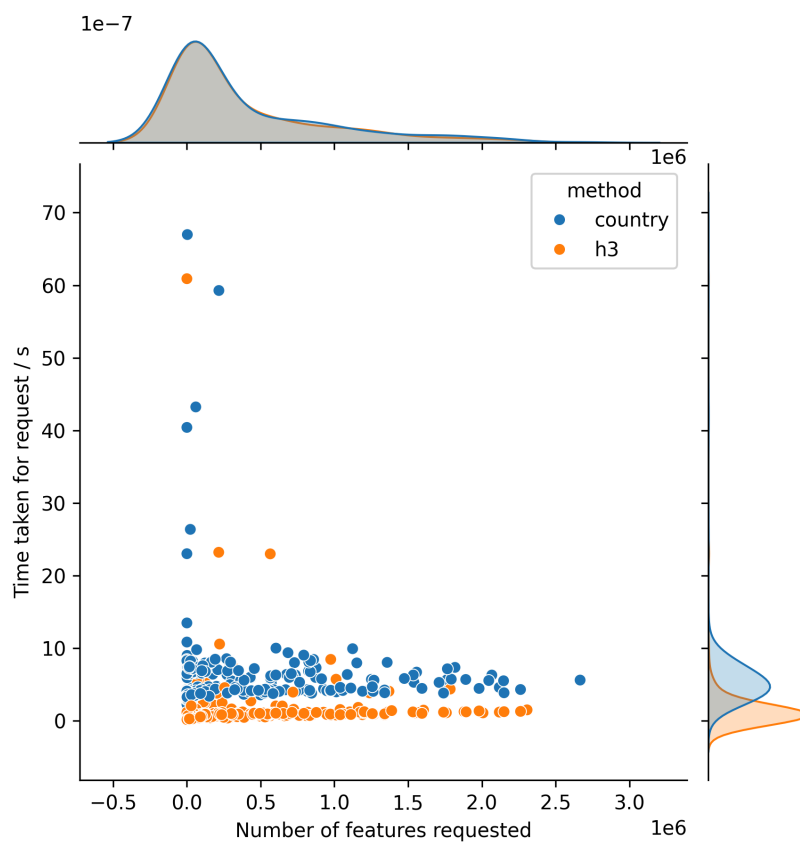


Figure 16: Graph showing time taken of query by the number of features requested